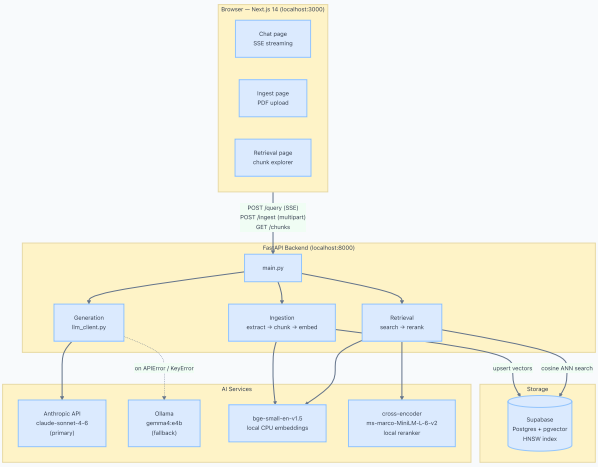
DocuMitra — System Architecture

RAG chatbot for private PDF corpora · Next.js 14 + FastAPI + pgvector + Anthropic

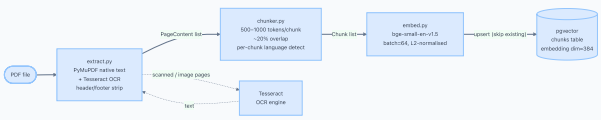
1. System Overview

DocuMitra ingests a private corpus of large PDFs, indexes them in a pgvector store, and answers natural-language questions with citations. The frontend streams answers in real-time via Server-Sent Events; the backend falls back from Anthropic to a local Ollama model when the primary API is unavailable.



2. Ingestion Pipeline

Each PDF is extracted page-by-page (native text via PyMuPDF; OCR via Tesseract for scanned pages and embedded images), split into overlapping token-bounded chunks, encoded with a local embedding model, and upserted into pgvector. Re-ingestion is idempotent — existing chunk IDs are skipped.



CHUNK METADATA STORED PER ROW

Field	Type	Description
`chunk_id`	text PK	SHA-256 of pdf_id + filename + page + index (deterministic)
`pdf_id`	text	SHA-256 of file content
`filename`	text	Original filename — used in citations

`page_number`	int	1-based page — used in citations
`text`	text	Chunk text (500–1000 tokens)
`language`	text	ISO 639-1 language code
`bbox`	jsonb	Bounding box (x0, y0, x1, y1) if available
`embedding`	vector(384)	L2-normalised bge-small-en-v1.5 vector

3. Query Pipeline

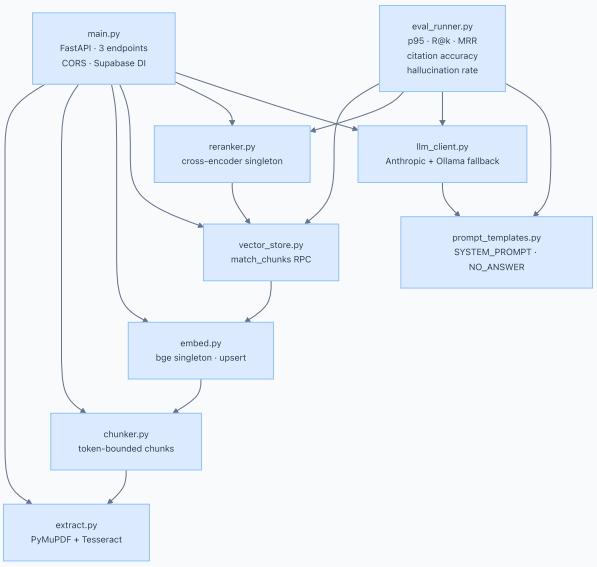
A user query passes through four stages: embedding, vector retrieval, optional cross-encoder reranking, and LLM generation. The answer is streamed back via SSE; every claim must be immediately followed by a citation in `[filename, p.N]` format.

LATENCY BUDGET (2–5 S TOTAL)

Stage	Target	Notes
Query embedding	< 100 ms	Model kept warm in memory (singleton)
HNSW retrieval (top-20)	< 200 ms	Supabase `match_chunks` RPC
Cross-encoder rerank	~ 500 ms	Optional — skip if over budget
LLM generation	1.5–3 s	Streamed to reduce perceived latency
Total	**2–5 s**	End-to-end including network

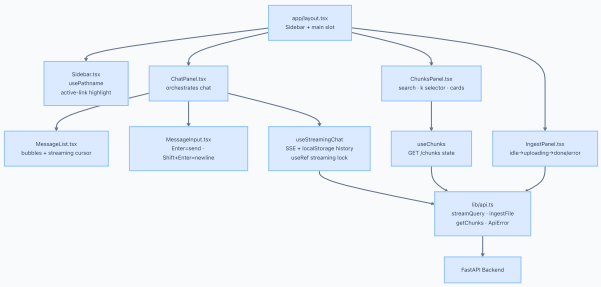
4. Backend Module Dependency Graph

All pipeline logic lives in focused single-responsibility modules. `main.py` is the only entry point — it wires the modules together via FastAPI route handlers.



5. Frontend Component Tree

The Next.js App Router shell renders a persistent sidebar alongside the active page. Each page imports one focused client component. All backend communication is centralised in `lib/api.ts`; business logic lives in two hooks.



6. Technology Stack

Layer	Technology	Purpose
Frontend	Next.js 14 · TypeScript · Tailwind CSS	Chat UI, ingestion, retrieval visualization
Backend	Python FastAPI · Uvicorn	REST + SSE API server
Vector DB	Supabase (Postgres + pgvector)	HNSW cosine similarity search
Embeddings	`BAAI/bge-small-en-v1.5` (sentence-transformers)	384-dim, CPU-friendly, free

Reranker	`cross-encoder/ms-marco-MiniLM-L-6-v2`	Cross-encoder top-20 → top-5 (optional)
LLM (primary)	Anthropic `claude-sonnet-4-6`	Answer synthesis with citations
LLM (fallback)	Ollama `gemma4:e4b`	Automatic fallback on APIError
PDF extraction	PyMuPDF + Tesseract	Native text + OCR for scanned pages
Testing	pytest (205 tests) · Jest + @testing-library/react (46 tests)	Full TDD coverage